FPL prediction model: project handover

Oct 16, 2026 · @Daljit Singh Johal

Purpose

This project predicts Fantasy Premier League (FPL) points for every player from free data, then picks a squad or suggests transfers within FPL's rules. It was built in an incognito chat that can't be saved, so this handover records what exists, why it was built that way, and what to do next.

To continue in a new chat, paste this document in first, then attach any scripts you want changed. Paste error messages in full, including the command you ran.

Project files

All files sit in one folder: `C:\Users\Daljit Johal\Documents\Other\FPL Modelling`. Python 3.11 or later, managed with Poetry 2.

File	What it does
`fpl_model.py`	Main script. Loads data, builds features, trains the model, forecasts the next 3 gameweeks, runs the optimiser, prints the squad and saves predictions and the plan. Holds `CONFIG` (model settings) and the command-line options.
`fpl_api.py`	Official FPL API: current-season player histories, prices, fixtures, your team's picks, history and transfers, and actual points for finished gameweeks. Caches in `fpl_api_cache/`.
`understat_loader.py`	Downloads Understat per-match player stats and shots, builds shot features, and matches Understat players and teams to FPL. Caches in `understat_cache/`.

File	What it does
`optimiser.py`	Integer programme (HiGHS via scipy) for a squad from scratch or for transfers, with a separate XI and captain each gameweek. Also formats the output tables.
`my_team.py`	Reads your squad, bank, selling prices and estimated free transfers from the public FPL API using your team ID.
`tune.py`	Walk-forward tuning. Compares settings on 2024-25, checks the winner on 2025-26, prints settings to paste into `CONFIG`.
`track.py`	Live tracking. Compares saved forecasts and suggested line-ups with actual points, and against FPL's own expected points.
`pyproject.toml`	Poetry project: dependencies, your team ID (`FPL_TEAM_ID`) and the stored `poe` commands. `package-mode = false`.
`poetry.lock`	Exact tested versions: pandas 3.0.6, scikit-learn 1.9.1, scipy 1.17.1, plus poethepoet as a dev tool.
`.gitignore`	Excludes `.venv/`, caches, `output/` and tuning results.

Outputs go to `output/`: `predictions_<season>_gw<N>.csv`, `plan_<season>_gw<N>.json` and `tracking_*.csv`.

Data sources

All data is free. Past seasons come from a GitHub archive, the current season from the live FPL API, and shot detail from Understat.

Source	Used for	Notes
vaastav/Fantasy-Premier-League	Training seasons 2022-23 to 2025-26: player-by-match points, minutes, xG, xA, xGC, BPS, ICT, saves, prices	2022-23 is the first season with FPL xG. Tackles, recoveries and CBI exist from 2025-26 only. The archive only had 2026-27 up to gameweek 1, so it isn't used for the current season.
Official FPL API	Current season (2026-27): player histories, prices, injury flags, fixtures, your team, actual points	`bootstrap-static/`, `fixtures/`, `element-summary/{id}/`, `entry/{id}/...`, `event/{gw}/live/`. No login needed. About 700 requests on a fresh download.
Understat	Shots, key passes, xG, xA, xGChain, xGBuildup, and shot-level features	JSON endpoints `getLeagueData/EPL/{year}` and `getMatchData/{id}`, with an `X-Requested-With` header. One-second delay between requests; about 7 minutes per season on first download. No tackles or saves.

FBref isn't used: its advanced stats (including xG) were removed in January 2026 after Opta ended its data supply. StatsBomb's free event data only covers Premier League 2003-04 and 2015-16, which is too old for current players.

How the model works

One gradient-boosted tree model (scikit-learn `HistGradientBoostingRegressor`) predicts each player's expected FPL points per fixture. It is trained with squared error, so it predicts the mean, which is what the optimiser needs.

Training rows. One row per player per match, 2022-23 onwards, with the actual points as the target. Inputs only use matches before that row (a shift of one), so the model never sees the result it's predicting.

Inputs (30 with Understat).

- 13 FPL form inputs: minutes, starts, total points, xG, xA, xGC, BPS, ICT, saves, defensive contribution, tackles, recoveries, CBI.

- 11 Understat form inputs: shots, key passes, xG, xA, xGChain, xGBuildup, non-penalty xG, penalty xG, shots in the box, headed shots, set-piece xG.
- 2 opponent inputs: the opponent's rolling xG for and against over its last 5 games.
- 4 player context inputs: games played this season, price, home or away, position.

Recency. Form is an exponentially weighted average with a half-life of 2 matches: the latest game counts fully, two games back half, four games back a quarter. About 82% of the weight falls on the last 5 matches. Form carries across seasons using FPL's `code` field, which stays the same for a player from season to season. Training rows from all seasons are weighted equally.

Training data vs form window. The training data teaches the model how stats turn into points; those relationships change slowly, so long history helps. The form window describes each player's current state; players change quickly, so recent games dominate. The window is calculated the same way in training and forecasting.

Positions. One shared model, with position as a category. The trees can split on it, but there are no separate weights per position. Separate models per position were tested and did worse (see Tuning).

Inputs with no data. Any input that is completely empty in the training data is skipped (for example tackles when every training match predates 2025-26). scikit-learn 1.9.1 refuses to train on an all-empty input.

Forecast. Each player's latest form is held fixed, while opponent and home or away change per fixture, for the next 3 gameweeks. Double gameweeks are summed, blanks are zero, predictions are floored at 0, then scaled by FPL's chance-of-playing percentage. Players who have left or are ineligible get 0.

Understat matching. Teams are matched by comparing which players appeared on each side on the same date and venue. Players are matched by name similarity within each match, using both the full FPL name and the display name (for nicknames such as Casemiro or Beto), with minutes as a tiebreak. Matching is one-to-one per match, then the most common match across a player's games wins. Match rates on the first run were 98.7% to 99.4% per season, before the nickname fix.

Tuning

The tuned model beat the original settings by about 1.4% and a simple last-5 average by about 8% on 2025-26. These results are from FPL data only, run in the build chat; rerun `tune.py` with Understat and paste its settings into `CONFIG`.

Method. Walk-forward: for gameweeks 6, 12, 18, 24, 30 and 36, train only on matches before that gameweek, then predict it. Settings are chosen on 2024-25 in four steps (form measure, training weights, shared vs per-position, tree settings), then checked once on 2025-26. The main measure is RMSE for regulars (players averaging 60+ minutes over their previous 3 games). RMSE rewards predicting the mean; MAE would reward the median, about 2 points for most regulars.

Current `CONFIG`: form `ewm`, half-life 2, carried across seasons, equal training weights, one shared model, learning rate 0.05, 300 trees, 15 leaves, at least 200 rows per leaf.

Results on 2025-26 (RMSE, points per player per match, regulars only; lower is better):

Settings	All regulars	GK	DEF	MID	FWD
Last-5 average (no model)	3.438	3.027	3.517	3.458	3.462
Original settings	3.193	2.709	3.245	3.256	3.217
Tuned, one shared model	3.148	2.673	3.253	3.151	3.178
Tuned, one model per position	3.181	2.744	3.290	3.155	3.281

On 2024-25, a half-life of 2 (RMSE 3.152) beat a 5-game rolling window reset each season (3.210). Down-weighting older training seasons didn't help. Differences of 0.02 to 0.03 may be noise, since each season is tested on 6 gameweeks.

Optimiser and transfer mode

The optimiser chooses 15 players and, for each of the next 3 gameweeks, its own XI and captain, maximising predicted points. It's an integer programme solved by HiGHS through `scipy.optimize.milp`, so no extra solver is needed.

Objective. Starters' points, plus the captain's points a second time, plus 10% of bench points (`BENCH_WEIGHT`) as cover for injuries.

Rules enforced. 2 GK, 5 DEF, 5 MID, 3 FWD; at most 3 per club; each week's XI has 1 GK, 3 to 5 DEF, 2 to 5 MID, 1 to 3 FWD; a starter must be owned, and the captain must start. Budget: £100.0m from scratch.

Transfer mode (`--team-id`). Solves once for each number of transfers from 0 to `--max-transfers` (default 3), forcing exactly that many changes. Each option shows points after hits (4 per transfer beyond the free ones), the gain over making none, and players out and in. The budget allows the bank plus selling prices of players sold.

Estimates you should check against the FPL app (both can be overridden with `--free-transfers` and `--bank`):

- Selling price: purchase price plus half of any rise, rounded down to £0.1m; a fall is passed on in full. Purchase prices come from your transfer history (ignoring Free Hit weeks), or the start-of-season price for original players.
- Free transfers: replayed from your history. One a week, up to 5 banked, unused ones carried over, and saved ones kept through a Wildcard or Free Hit. How chip weeks affect the count is the least certain part.

The squad is read from the last deadline, so run it before making that week's transfers. A Free Hit week is skipped in favour of the squad before it.

Live tracking

`track.py` scores each saved forecast against actual points once its gameweek finishes. Tracking starts from the first run of the current `fpl_model.py`; files saved by earlier versions have no player IDs and are ignored.

Each `fpl_model.py` run saves a predictions file (every player, with IDs, the run time, the deadline and FPL's own `ep_next` as a benchmark) and a plan file (suggested squad, line-ups, captains, transfers). Forecasts saved after their deadline are ignored.

The report covers "likely starters" (predicted 2+ points) and shows:

- Accuracy by horizon (1, 2 and 3 weeks ahead): MAE, RMSE, bias (above zero means predicted too high) and correlation.
- Model vs FPL's `ep_next`, one week ahead, on the same players. If the model can't beat this, it needs work.
- Accuracy by position and by gameweek.
- Calibration: players grouped by predicted points, next to what they actually averaged.
- Team points: the suggested line-up's predicted and actual points, next to your own team's, with hits shown separately. Your actual points include automatic substitutions; the suggested line-up's don't.

Judge trends over 5 to 10 gameweeks; one captain haul can swing a week by 20 points.

Setup and weekly routine

Every week, before the deadline and before making transfers, run `poetry run poe weekly`. It scores last gameweek, then suggests transfers.

One-off setup (PowerShell, in the project folder):

1. Install Python 3.11+ from python.org, ticking "Add Python to PATH".
2. Install Poetry: `python -m pip install --user pipx`, then `python -m pipx ensurepath`, then `pipx install poetry`. Reopen PowerShell.
3. Put your team ID in `pyproject.toml` (`FPL_TEAM_ID = "..."`). It's the number after `/entry/` in your FPL Points page address.
4. Run `poetry lock`, then `poetry install`.
5. Run `poetry run poe` to check the commands are listed.
6. Run `poetry run poe tune` and paste the printed settings into `CONFIG` in `fpl_model.py`.

Stored commands (`poetry run poe <task>`):

Task	What it does	When
`weekly`	`track`, then `transfers`	Before each deadline
`transfers`	Transfer options, line-up, captain, predicted points per player and per gameweek	As needed
`track`	Tracking report	After a gameweek finishes
`wildcard`	Best squad from scratch within £100.0m	Wildcard weeks
`tune`	Walk-forward tuning	Every 4 to 6 weeks, or after rule changes
`quick`	Transfers without Understat	If Understat is down
`refresh`	Deletes cached FPL data	If data looks stale

Options can follow a task, for example `poetry run poe transfers --free-transfers 2 --bank 1.5`. Without `poe`, the equivalent is `poetry run python fpl_model.py --team-id <ID>`.

Weekly checks. Run it at least a few hours after the last match, so FPL has finalised points. Check the forecast starts at the next gameweek, the free transfers and bank match the app, and Understat match rates stay above 95%. Make all changes in the FPL app yourself; the scripts never change your team.

Common errors.

- "No module named ..." or "'poe' is not recognized": the command is missing `poetry run`, or `poetry install` didn't finish. Run `poetry lock` then `poetry install`.
- "pyproject.toml changed significantly": run `poetry lock`, then `poetry install`.
- FPL API errors: FPL updates around deadlines and after matches; wait an hour.
- Understat errors: use `poetry run poe quick` that week.
- Odd import errors after editing: check for typos (one earlier error was `sklearn.enemble`), or re-download the file.

Known limitations

The model's error is about 3 points per regular player per match, so gains of a few points between transfer options are a tie.

- **Minutes.** Whether a player starts is the largest source of error; there's no separate minutes model yet.
- **Later weeks.** Gameweeks 2 and 3 of the forecast hold current form fixed, so they're less accurate than the tuning numbers, which test one week ahead.
- **One-week planning.** Transfer mode decides only this week's moves. It doesn't value rolling a free transfer, plan across weeks, time chips, or account for price changes.
- **Opponent strength.** Based on 5-game rolling xG, which is noisy and slow to react.
- **Rule changes.** 2026-27 rebalanced the bonus points system; training seasons reflect the older rules.
- **Untested against live services from the build chat.** The FPL API and Understat couldn't be reached from the build environment; code was tested on mock responses built from real archive data. Your runs are the real test.
- **Estimates.** Selling prices and free transfers are estimated from public data; check them each week.
- **Tuning without Understat.** Current `CONFIG` came from FPL data only until you rerun `tune.py`.

Improvement roadmap

Next: build the betting odds inputs, starting by confirming a free source of odds for upcoming fixtures, then measure them against the current version using `track.py` for a few weeks. Live tracking (step 1) is done.

#	Improvement	Why	Effort
1	Live tracking	Measures every later change on this season's data	Done
2	Betting odds for team strength (next)	Better clean-sheet and attacking predictions than rolling xG. Historical odds free at football-data.co.uk; free upcoming odds still to find (for example The Odds API free tier)	Medium
3	Minutes model: chance of starting and of 60+ minutes, then points if playing	Largest source of error. Inputs: recent starts and minutes, FPL news flags, days of rest, European fixtures, new signings in the same position	Medium
4	Predict each scoring event separately, then apply FPL scoring rules by position	Handles position scoring properly and adapts to rule changes; reuses the minutes model	Large
5	Multi-week transfer planner	Weighs using a free transfer now against rolling it; chip timing later	Large
6	Separate models for 1, 2 and 3 weeks ahead	Honest predictions for later weeks	Medium
7	More player inputs: penalty and set-piece duty (already in FPL data), share of team xG and shots, Understat history from other leagues for new arrivals, home/away opponent splits	Cheap gains	Small
8	Predict a range of outcomes, not just the average	Captaincy and differential picks	Medium

#	Improvement	Why	Effort
9	Schedule the weekly run with Windows Task Scheduler	Consistent tracking, no manual step	Small

Problems already fixed

These are recorded so they aren't reintroduced.

Problem	Fix
Archive had only gameweek 1 of 2026-27, so forecasts were out of date	`fpl_api.py` rebuilds the current season from the live API in the archive's format (`LIVE_CURRENT = True`)
Archive files read as Latin-1, garbling accented names	Read as UTF-8
PuLP 4 had no bundled solver	Optimiser switched to `scipy.optimize.milp` (HiGHS, no extra install)
Understat team matching failed early in the season, when many games share a date	Teams matched by player overlap on the same date and venue
Two players with the same first name (Arsenal's Gabriels) could swap	One-to-one matching per match, plus Jaccard scoring on name tokens
"Rodri" didn't match "Rodrigo Hernández"	Name tokens of 4+ letters match on prefix
Nickname players unmatched (Casemiro, Beto, Fabinho, Jorginho, Vitinho)	Matching also uses FPL's display name
scikit-learn 1.9.1 errored on all-empty inputs (tackles before 2025-26)	Inputs with no values in the training data are skipped
`python tune.py` couldn't find sklearn	Commands must start with `poetry run`
Lock file out of date after updating `pyproject.toml`	`poetry lock`, then `poetry install`